Understanding Basic TypeScript Types

At its most basic, TypeScript allows you to annotate any variable declaration with a type by using the syntax `: <type>`, as in `let x: number`. As we'll see, this can get more complicated, but the starting point is annotating variables with types.

TypeScript defines four basic types:

- `boolean`: A Boolean value must be either JavaScript's `true` or `false` value.
- `number`: JavaScript only has one numeric type for floating point numbers. TypeScript's number type supports floating point and integer literals, hex literals (`0xab32`), octal literals (`0o1234`), and binary literals (`0b10010`).
- `string`: TypeScript allows both single and double quotes as string delimiters and supports the backquote (\) syntax for template strings.
- `object`: TypeScript defines an object as anything that is a value and is not one of the previous types, so not just raw objects, but also all instances of classes. Normally, you'd use a more specific type annotation as described in the following, rather than using plain `object`.

Both `null` and `undefined` are also TypeScript types, and you can say something like `let z: null = null`, though it's not clear why you'd want to.

By default, TypeScript allows the values `null` and `undefined` to be assigned to any variable no matter what its declared type is. If you are familiar with other relatively modern static-typed languages like Elm, Rust, or Swift, you may know that those languages force you to explicitly declare when `null` is a valid value for a given variable. That is not the default case in TypeScript, presumably because forcing explicit null declarations would make dealing with existing external JavaScript libraries quite complicated. Also, allowing `null` and `undefined` values makes it easier to gradually add TypeScript to an existing codebase. However, there is a compiler option, `--strictNullChecks`, which prohibits assigning `null` to a